

Classifying Composers from MIDI: A Comparison of LSTM and CNN Models

Marston Ward, Josue Sandoval, and Saman Tavasoli

University of San Diego

AAI-511: Neural Networks and Deep Learning

Professor Andrew Van Benschoten

July 28, 2026

Abstract

This paper compares two deep learning models that classify classical MIDI files as works by Bach, Beethoven, Chopin, or Mozart. We removed unreadable files and exact duplicates. We then placed each file into a fixed training, validation, or test split. Movements from the same work were not kept together. The long short-term memory (LSTM) and convolutional neural network (CNN) models use sequences of pitch, velocity, note duration, and time between notes. Class weights reduce the effect of an uneven class distribution. We addressed overfitting with weight decay and validation-based early stopping. In one seeded run, the CNN reached .822 validation accuracy and .812 weighted F1. The LSTM reached .759 accuracy and .748 weighted F1. The CNN may have benefited from learning short local note patterns, but one run cannot prove why it performed better. Even after addressing overfitting, the CNN reached .982 training accuracy compared with .822 validation accuracy. This gap shows that substantial overfitting remained. The controls were likely not strong enough: the 106,916-parameter CNN had only 1,125 training files, dropout was applied only after pooling, no data augmentation was used, and training reached the 20-epoch limit before early stopping could trigger. The CNN was the stronger model on this validation split, but the results do not yet show reliable performance on new musical works. Future work should group files by musical work, repeat the experiment, and evaluate the final models once on the untouched test set.

Keywords: composer classification, MIDI, deep learning, LSTM, CNN, music information retrieval

Classifying Composers from MIDI: A Comparison of LSTM and CNN Models

Automatic composer identification asks a simple question: who wrote a piece of music? MIDI files store notes, timing, and duration directly. Unlike audio, they do not include differences caused by singers, instruments, microphones, or recording rooms. This makes MIDI useful for models that learn musical sequences and patterns.

Related Work

Hochreiter and Schmidhuber (1997) introduced the long short-term memory architecture, which can learn relationships across a sequence. This makes it suitable for ordered musical notes. LeCun et al. (2015) describe convolutional networks, which learn short local patterns. Kingma and Ba (2015) introduced Adam, the optimizer used to train both models.

Earlier studies often combine selected musical features with statistical or machine-learning models. Conklin (2013) showed that several views of a musical piece can improve classification. McKay et al. (2018) presented jSymbolic, a tool for extracting features from symbolic music. Our models learn from note sequences instead. We use summary features only for exploratory data analysis (EDA).

Method

Dataset

This project uses the *MIDI Classic Music* dataset from Kaggle (Fedorak, n.d.). Kaggle lists it as Version 1 with an unknown license. The dataset contains MIDI files from many composers. We kept files by Bach, Beethoven, Chopin, and Mozart. The helper script `src/prepare_dataset.py` downloads the archive (when `~/.kaggle/kaggle.json` is configured) and extracts these four composer folders. It also creates filtered copies for inspection, but the models do not use those copies.

The modeling pipeline scans the raw composer folders. It removes unreadable files and exact duplicates. It then uses a fixed random seed to create a stratified 70/15/15 split

for training, validation, and testing. The saved split is reused unless the user asks to rebuild it. This keeps file assignments the same between runs. However, the split does not group files by musical work, catalog number, or collection. For example, movements of Mozart’s Symphony No. 40, K. 550 appear in all three splits. The validation score may therefore be too high for truly new works. The master notebook reports the class counts.

After cleaning, 1,608 MIDI files remain. The training split has 1,125 files, the validation split has 241, and the test split has 242. We removed 22 malformed or duplicate files. Bach makes up about 63% of the training data (709 files), while Chopin makes up 8% (92 files). Both models use class-weighted cross-entropy loss to reduce bias toward Bach. We report validation accuracy, precision, recall, F1 scores, and confusion matrices. We do not use the test split during model development.

Computational Workflow

Pixi provides a locked Python 3.12 environment, and the saved manifest keeps file assignments fixed. After `pixi install`, users run `pixi run prepare-dataset` to get the data and `pixi run notebook` to open the master notebook. The code can run on supported hardware accelerators or a CPU, which makes the workflow portable across systems. The environment and preprocessing steps can be repeated. Exact model scores may still change because PyTorch is not forced to use deterministic algorithms, and batch size can differ across systems. The notebook runs the workflow and presents the results. Reusable code is stored in `src/`.

The first run saves EDA data to `dataset/processed/eda_metadata.csv`. Later runs load this file instead of reading every MIDI file again. Users must refresh the file after changing the data or split because the code does not check it automatically.

Exploratory Data Analysis

Before training, we measure duration, tempo, note count, note density, pitch range, and mean pitch. We summarize only the training files, so validation and test files do not shape the analysis. Table 1 shows the mean values for each composer.

Table 1*Composer-Level Feature Summary (Train Split)*

Composer	Duration (s)	Note density (notes/s)	Pitch range (semitones)	Tempo (BPM)
Bach	160.3	9.3	43.1	89.7
Beethoven	518.3	14.5	61.1	123.3
Chopin	222.3	11.6	63.0	112.0
Mozart	415.5	13.7	55.3	104.9

Note. Values are per-composer means computed from cached, parsed MIDI metadata. Unreadable files and exact duplicates are excluded by the clean manifest.

Three patterns helped guide feature selection. First, note density differs by composer. Bach averages 9.3 notes per second, while Beethoven averages 14.5. Raw note count is also tied to file length. Second, Bach has a smaller mean pitch range ($M = 43.1$ semitones) than Beethoven and Chopin ($M \approx 62$). These differences may come from instrumentation, arrangements, data sources, or MIDI encoding, not only from composing style. Third, tempo and velocity may reflect how a MIDI file was created rather than how the music was performed. We therefore treat these values with care.

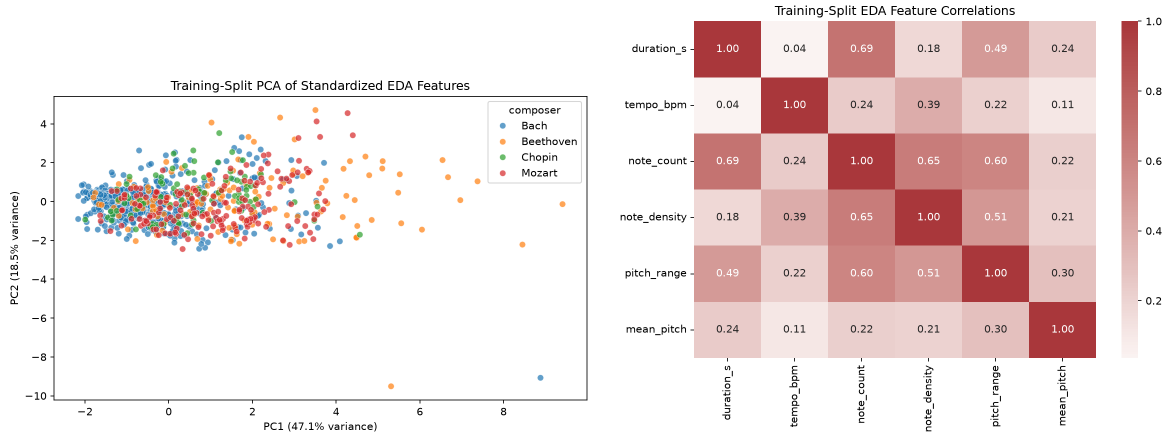
Principal component analysis (PCA) showed strong overlap between composers in the first two components. This led us to test models that keep note order and timing. However, the PCA plot leaves out 34.4% of the standardized variance, and we did not test a classifier based only on summary features. The plot therefore does not prove that summary features cannot work.

The first PCA component explains 47.1% of the variance, and the second explains 18.5%. Together they explain 65.6%, but the composers still overlap in the plot. Note count and duration are also related ($r = .69$). For this reason, we use note density and sequence features instead of relying only on total note counts. Figure 1 shows both

analyses, which use training data only.

Figure 1

Training-split PCA scores (left) and EDA feature correlations (right).



Preprocessing and Feature Extraction

For both models, notes are sorted by start time, pitch, and end time. Each note has four values: pitch, velocity, duration, and time since the previous note. Pitch uses values from 1 to 128, while zero marks padding. Every sequence is cut or padded to 4,096 notes. We calculate the mean and standard deviation for the continuous features from nonpadding training notes only. We then use those same values for validation and test data. This prevents information from those splits from entering the feature preparation step.

Model Architectures

LSTM Model

The LSTM changes each pitch into a 32-value embedding and joins it with the three continuous features. A one-layer LSTM with 64 hidden units processes the sequence. The model uses the state at the last real note, so padded notes do not choose the prediction. It then applies 30% dropout and a four-class output layer.

CNN Model

The CNN uses the same pitch embedding and continuous features. It has three convolution blocks with 64, 128, and 128 filters. The first two blocks use five-note kernels and four-note max pooling. The third uses a three-note kernel. Each convolution is followed by batch normalization and ReLU. Global max pooling creates one representation for the file. The model then applies 30% dropout and a four-class output layer. Both models use the same files, normalization, class weights, and validation-based selection process.

Training

Both baselines are trained for up to 20 epochs with Adam (Kingma & Ba, 2015) at a learning rate of .001 and weight decay of 1×10^{-4} . Class weights are $w_c = N/(Kn_c)$, where N is the number of training files, K is the number of classes, and n_c is the number of files in class c . Gradients are clipped at 1.0.

After each epoch, the model is tested on the validation split. We save the model with the highest validation accuracy, using lower validation loss to break a tie. Training stops after five epochs without a new best model. In this run, CNN training accuracy reached 96.8% at epoch 20, while validation loss varied. The LSTM validation loss began to rise after epoch 10 even as training loss fell. These patterns show that overfitting remained. We did not run a separate test to measure the effects of weight decay or early stopping. We also did not use data augmentation or evaluate the test split.

Results

The notebook evaluated both models on the same 241 validation files. In this seeded run, the CNN scored 6.2 percentage points higher in accuracy and 6.4 points higher in weighted F1. Table 2 shows the selected models. The 242 test files were not used.

As Figure 2 shows, the LSTM stopped after epoch 11 and kept its epoch-6 model. The CNN completed all 20 epochs and kept its epoch-18 model. The saved CNN reached 98.2% accuracy on training data but only 82.2% on validation data. This 16-point gap shows strong overfitting.

Table 2*Validation Results for Selected Baselines*

Model	Epoch	Accuracy	Weighted F1	Macro F1	Test evaluation
LSTM baseline	6	.759	.748	.622	Reserved
CNN baseline	18	.822	.812	.710	Reserved

Note. Metrics are from validation-selected checkpoints trained with weight-decayed Adam on the fixed clean manifest. The test split was not evaluated.

Figure 3 shows results for each composer. Compared with the LSTM, the CNN improved recall for Chopin from .650 to .800 and for Beethoven from .258 to .613. Both models had more difficulty with Beethoven and Mozart than with Bach.

Discussion

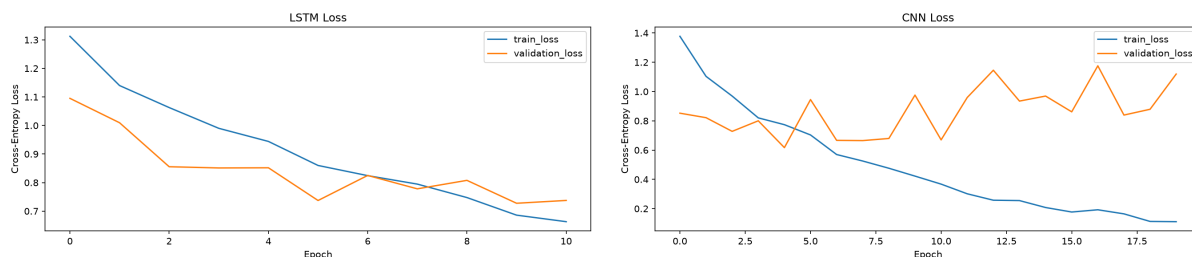
The CNN performed better on the validation split, but this does not prove that it will perform better on new works. The LSTM learns the order of the full sequence. The CNN learns short ordered patterns and then uses global max pooling, which removes their exact positions. Both models use note duration, velocity, and timing that are missing from the EDA summaries.

We used class weights because Bach dominates the training data. However, we did not train an unweighted model, so we cannot measure how much the weights helped. In the same way, we cannot measure the separate effects of weight decay and early stopping. These methods may have limited overfitting, but they did not remove it. The CNN still has about a 16-point gap between training and validation accuracy.

Several factors may explain why overfitting remained. The CNN has 106,916 parameters but only 1,125 training files. Its 30% dropout is applied only after global pooling, so the convolution blocks receive no dropout. The training data were not augmented. Early stopping also did not stop the CNN: its best model was at epoch 18, and the run reached the 20-epoch limit before the five-epoch patience period ended. These

Figure 2

Training and validation loss for the LSTM (left) and CNN (right).



points are possible explanations, not proven causes, because we did not run controlled comparisons. Choosing the best epoch from validation results can also make the reported validation score look slightly better than it would on new data. A final test score is therefore still needed.

The CNN may have done better because its filters can find short note patterns. However, the CNN and LSTM also differ in size, normalization, pooling, and how they handle padding. More runs and focused experiments are needed to find the real cause.

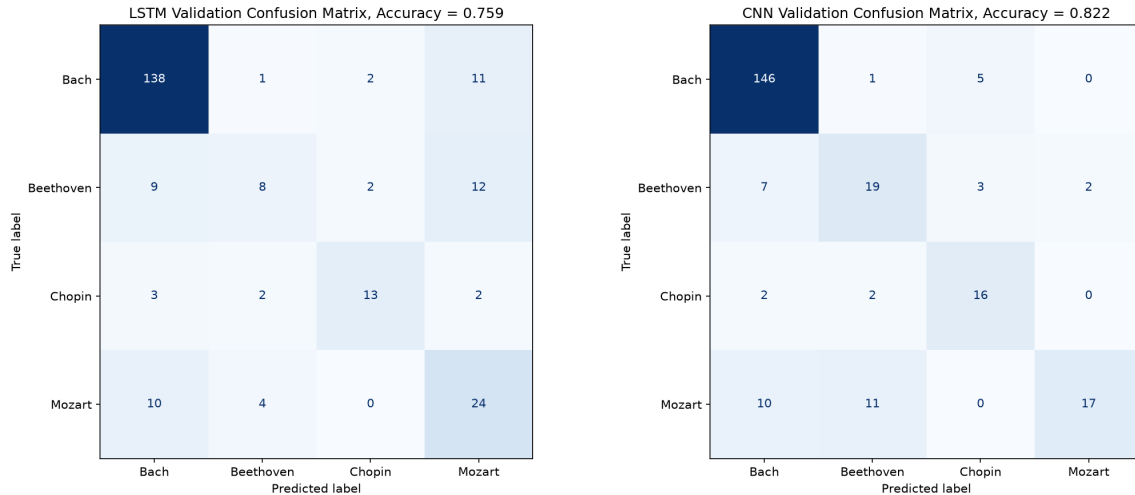
The comparison is fair within this experiment because both models use the same files, features, class weights, and selection process. Still, related movements can appear in different splits. A final test on the current split would measure performance on held-out files, not necessarily on completely new musical works. The results also come from one seeded run. Repeating the experiment would show how much the scores change from run to run.

Conclusion

This project builds a fixed file-level MIDI pipeline and compares LSTM and CNN models. Pixi locks the software environment, the saved manifest keeps file assignments fixed, and the notebook runs reusable code from `src/`. The CNN scored higher in this validation run, but it also showed strong overfitting. It is therefore the better model for this split, not a proven better choice for all composer-classification tasks. Weight decay and early stopping addressed the problem, but they were not enough to remove it. The likely

Figure 3

Validation confusion matrices for the LSTM (left) and CNN (right).



reasons include the CNN's size compared with the training set, dropout only after pooling, no data augmentation, and a training limit that ended before early stopping could stop the CNN. Controlled comparisons are needed to confirm the effect of each factor.

The most important improvement is to group files from the same work or collection before splitting the data. This would keep related movements in one split. The experiment should then be repeated with several seeds, deterministic PyTorch settings, and the same batch size across systems. Results should include the average score and the amount of variation between runs.

Future experiments should test one design choice at a time. These tests should cover class weights, weight decay, early stopping, model size, and CNN pooling. The CNN should also mask padded notes before global pooling so that file length does not affect its representation. If overfitting remains, a smaller CNN, stronger dropout, and careful pitch, tempo, or timing changes could be tested. All choices should use validation data only. Once the design is fixed, the final models should be evaluated once on an untouched test set grouped by musical work.

References

- Conklin, D. (2013). Multiple viewpoint systems for music classification. *Journal of New Music Research*, 42(1), 19–26. <https://doi.org/10.1080/09298215.2013.767040>
- Fedorak, B. (n.d.). *MIDI Classic Music* (Version 1) [License listed as unknown]. Kaggle. Retrieved July 28, 2026, from <https://www.kaggle.com/datasets/blanderbuss/midi-classic-music>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations, ICLR 2015*.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- McKay, C., Cumming, J. E., & Fujinaga, I. (2018). jSymbolic 2.2: Extracting features from symbolic music for use in musicological and MIR research. *Proceedings of the 19th International Society for Music Information Retrieval Conference*, 348–354.